

实验三：LZW 编解码实验

姓名：梁信、高居博、赵夕涵、刘嘉乐、黎玥含、赵欣瑶、罗雅丹、蓝浩宇

第 5 组

一、实验目标

本实验以 8 位灰度图像 `b8gray.bmp` 为输入，完成基于 LZW (Lempel–Ziv–Welch) 的无损图像编解码。实验目标如下：

1. 理解 LZW 字典编码的基本思想，掌握由像素序列生成压缩码流的方法；
2. 编写编码程序，以原灰度图像 I_s 为输入，输出压缩文件 `C.dat`；
3. 编写解码程序，以 `C.dat` 为输入，恢复解码图像 I_d ；
4. 计算编码后的 `bpp` 值和原灰度图像的信息熵；
5. 对 I_s 与 I_d 进行逐像素比较，验证 LZW 编解码的无损性。

二、实验原理

2.1 LZW 编码思想

LZW 是一种典型的自适应字典压缩算法。对于 8 位灰度图像，初始字典包含全部单字节灰度值：

$$\mathcal{D}_0 = \{0, 1, 2, \dots, 255\}. \quad (1)$$

编码时按行扫描图像像素，持续寻找字典中已经出现的最长序列 w 。若新序列 $w + k$ 尚未出现在字典中，则输出 w 的编码，并将 $w + k$ 加入字典。随着编码进行，重复纹理、相邻灰度结构和局部模式会被更长的字典项表示，从而减少码字数量。

2.2 LZW 解码思想

解码端采用与编码端相同的初始字典，并根据接收到的码字同步重建字典。设上一条解码序列为 w ，当前码字对应序列为 s ，则每读取一个新码字后，把

$$w + \text{first}(s) \quad (2)$$

加入字典。对于 LZW 中常见的特殊情形，即当前码字刚好等于下一条待加入的字典索引，解码序列取为

$$s = w + \text{first}(w). \quad (3)$$

只要编码和解码的字典更新规则一致，最终恢复的像素序列应与原图完全相同。

2.3 bpp 与熵

编码后每像素比特数 (bits per pixel, `bpp`) 用于衡量压缩表示的平均码长。本文同时给出仅统计 LZW 码流的 `bpp` 和包含文件头、调色板等元数据的 `C.dat` 文件 `bpp`：

$$\text{bpp}_{\text{code}} = \frac{N_{\text{code}} \times 16}{W \times H}, \quad \text{bpp}_{\text{file}} = \frac{8 \times \text{size}(\text{C.dat})}{W \times H}. \quad (4)$$

原灰度图像的信息熵定义为

$$H(I_s) = - \sum_{k=0}^{255} p_k \log_2 p_k, \quad (5)$$

其中 p_k 表示灰度级 k 在原图中的出现概率。熵反映图像灰度分布的不确定性，是无损编码平均码长的重要参考下界。

三、方法描述

3.1 编码框图

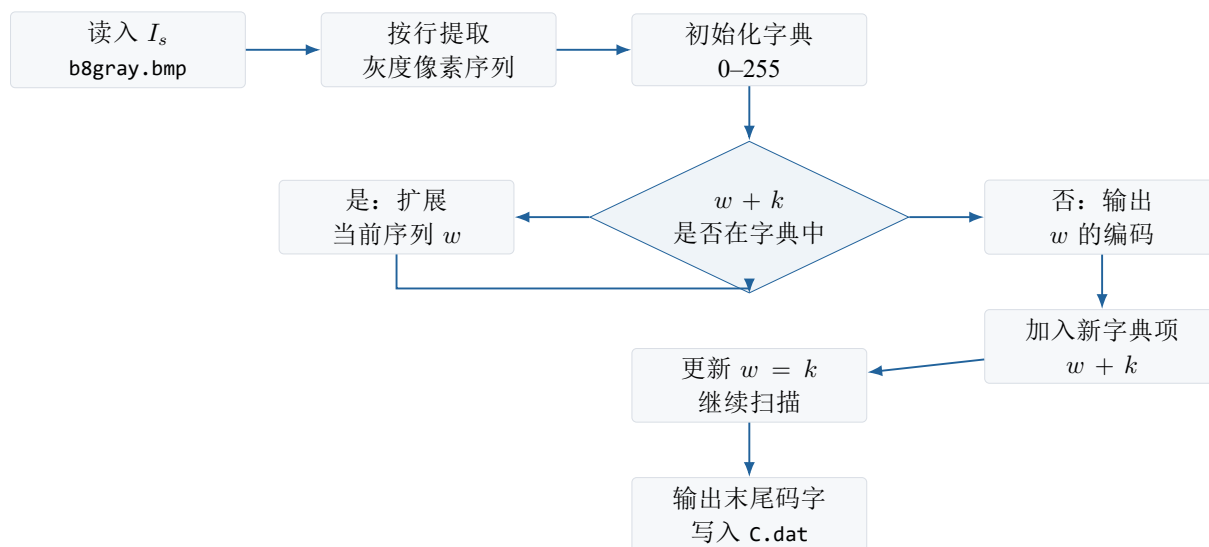


图 1: LZW 编码流程图

3.2 编码代码

编码程序位于 Task3/main.cpp。核心函数包括：

- `GetImageBytes`: 从 `HXLBMPFILE` 中提取 8 位灰度像素序列；
- `LzwEncode`: 按照 LZW 字典规则输出 16 bit 码流；
- `SaveCompressedFile`: 将图像尺寸、调色板、码流数量和码字写入 `C.dat`；
- `CalculateEntropy`: 统计原图灰度概率并计算信息熵。

3.3 解码框图

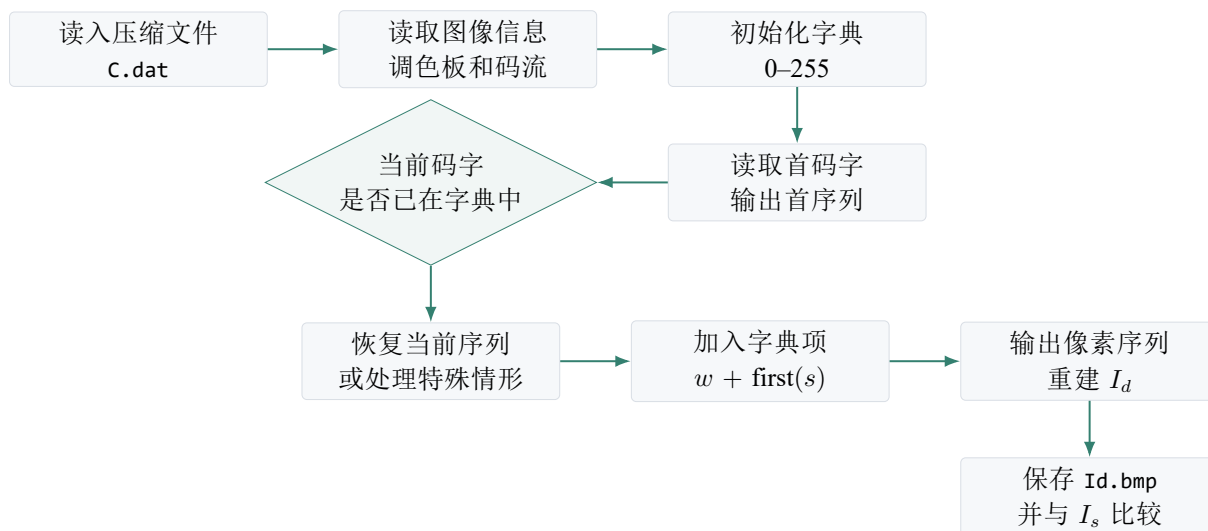


图 2: LZW 解码流程图

3.4 解码代码

解码程序同样位于 Task3/main.cpp。核心函数包括：

- LoadCompressedFile: 读取 C.dat 中保存的图像元数据、调色板和 LZW 码流；
- LzwDecode: 按 LZW 解码规则恢复原始像素序列；
- PutImageBytes: 将解码像素写回 HXLBMPPFILE 图像对象；
- SameImage: 比较原图和解码图的尺寸、通道数及全部像素值。

四、实验结果

4.1 图像恢复结果

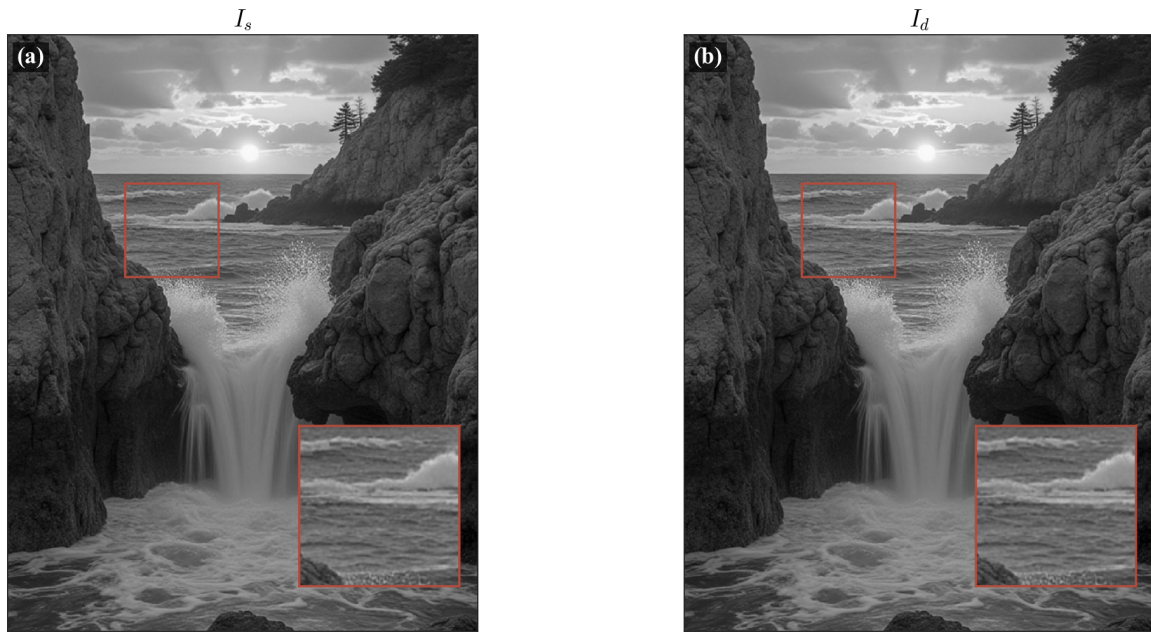


图 3: (a) 原图 I_s (b8gray.bmp) 与 (b) 由 C.dat 解码得到的图像 I_d 。红色矩形区域为放大对比细节。

从视觉上看，解码图像与原图完全一致。图 4 进一步给出了局部细节与残差分布验证：左、中两幅为同一区域的原图与解码图裁剪结果，右侧为全图像素残差 $|I_s - I_d|$ 的分布。残差直方图中所有像素均集中在 0，且 $\max \Delta = 0$ 、 $N_{\Delta > 0} = 0$ 、 $MSE = 0$ ，说明 LZW 编解码实现了严格无损恢复。

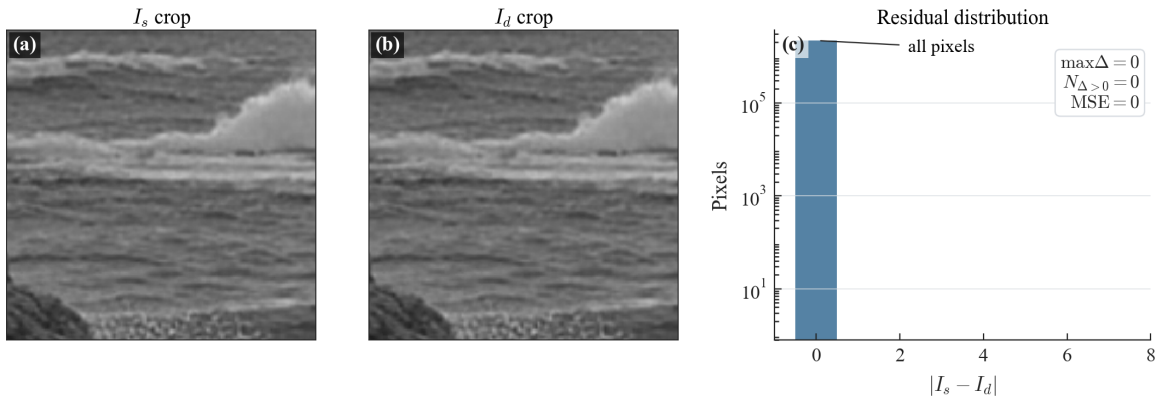


图 4: I_s 与 I_d 的局部裁剪及残差分布验证。(a) 原图局部区域；(b) 解码图同一区域；(c) 全图残差 $|I_s - I_d|$ 的像素数分布。残差全部位于 0，验证了解码结果与原图逐像素一致。

4.2 原图灰度分布

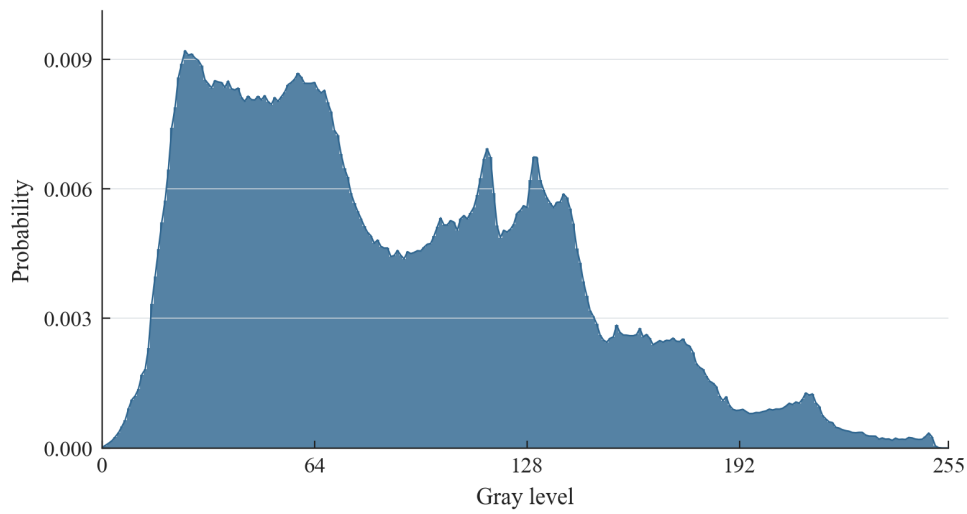


图 5: 原图 I_s 的归一化灰度直方图

原图灰度主要集中在中低灰度区间，同时覆盖了较宽的灰度范围。该分布决定了图像熵较高，但局部像素序列仍存在可被 LZW 字典利用的重复结构。

4.3 bpp、熵与同一性验证

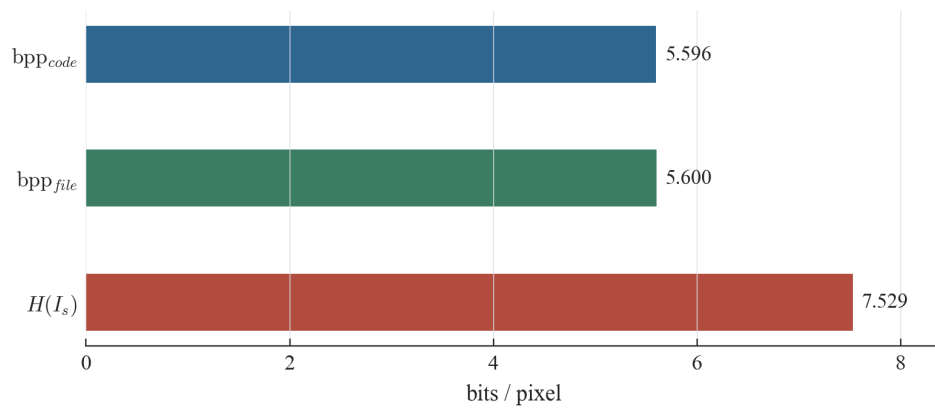


图 6: LZW 编码 bpp 与原图信息熵 $H(I_s)$ 对比。 bpp_{code} 和 bpp_{file} 均低于信息熵，说明 LZW 字典有效利用了像素序列的冗余。

项目	结果
输入图像	b8gray.bmp
图像尺寸	1314 × 1666
原始像素数	2,189,124
LZW 码字数量	765,705
编码后 bpp（仅 LZW 码流）	5.596430 bit/pixel
编码后 bpp（包含 c.dat 文件头）	5.600275 bit/pixel
原灰度图像熵	7.528544 bit/pixel
I_s 与 I_d 是否相同	YES

表 1: 实验三运行结果汇总

4.4 输出文件

文件	类型	说明
C.dat	压缩文件	编码程序输出，包含图像元数据、调色板与 LZW 码流
Id.bmp	解码图像	解码程序由 C.dat 恢复得到
result.txt	文本结果	保存 bpp、熵和同一性验证结果
figures/*.png	报告图片	报告中插入的图像、直方图和指标图

表 2: 实验输出文件说明

五、结论

本实验完成了灰度图像的 LZW 编码与解码。程序以 b8gray.bmp 为输入生成 C.dat，再由该压缩文件恢复出 Id.bmp。逐像素比较结果表明 I_s 与 I_d 完全一致，验证了算法实现的无损性。实验测得仅码流 bpp 为 5.596430 bit/pixel，包含文件头后的 C.dat bpp 为 5.600275 bit/pixel，均低于原图单像素灰度熵 7.528544 bit/pixel，说明 LZW 字典有效利用了图像像素序列中的上下文重复结构。

六、附录：函数代码

以下为除 HXLBMPFILE 类代码外的实验三主要代码。

Listing 1: 实验三 LZW 编解码完整实现

```
1 #include "../Task2/hxlbmpfile.h"
2
3 #include <algorithm>
4 #include <cmath>
5 #include <cstdint>
6 #include <cstdlib>
7 #include <fstream>
8 #include <iomanip>
9 #include <iostream>
10 #include <stdexcept>
11 #include <string>
12 #include <unordered_map>
13 #include <vector>
14
15 static const uint32_t kMaxCode = 65535;
16
```

```

17 struct EncodedFile {
18     int width = 0;
19     int height = 0;
20     std::vector<RGBQUAD> palette;
21     std::vector<uint16_t> codes;
22 };
23
24 static uint64_t MakeKey(uint32_t prefix, BYTE value) {
25     return (static_cast<uint64_t>(prefix) << 8) | value;
26 }
27
28 static void WriteU16(std::ofstream& out, uint16_t value) {
29     out.put(static_cast<char>(value & 0xff));
30     out.put(static_cast<char>((value >> 8) & 0xff));
31 }
32
33 static void WriteU32(std::ofstream& out, uint32_t value) {
34     out.put(static_cast<char>(value & 0xff));
35     out.put(static_cast<char>((value >> 8) & 0xff));
36     out.put(static_cast<char>((value >> 16) & 0xff));
37     out.put(static_cast<char>((value >> 24) & 0xff));
38 }
39
40 static uint16_t ReadU16(std::ifstream& in) {
41     uint16_t b0 = static_cast<unsigned char>(in.get());
42     uint16_t b1 = static_cast<unsigned char>(in.get());
43     if (!in) throw std::runtime_error("Unexpected end of file while reading uint16.");
44     return static_cast<uint16_t>(b0 | (b1 << 8));
45 }
46
47 static uint32_t ReadU32(std::ifstream& in) {
48     uint32_t b0 = static_cast<unsigned char>(in.get());
49     uint32_t b1 = static_cast<unsigned char>(in.get());
50     uint32_t b2 = static_cast<unsigned char>(in.get());
51     uint32_t b3 = static_cast<unsigned char>(in.get());
52     if (!in) throw std::runtime_error("Unexpected end of file while reading uint32.");
53     return b0 | (b1 << 8) | (b2 << 16) | (b3 << 24);
54 }
55
56 static std::vector<BYTE> GetImageBytes(HXLBMPFILE& bmp) {
57     if (bmp.iYRGBnum != 1) {
58         throw std::runtime_error("This experiment expects an 8-bit grayscale BMP.");
59     }
60
61     std::vector<BYTE> data;
62     data.reserve(static_cast<size_t>(bmp.iImagew) * bmp.iImageh);
63     for (int y = 0; y < bmp.iImageh; ++y) {
64         BYTE* row = bmp.pDataAt(y);
65         data.insert(data.end(), row, row + bmp.iImagew);
66     }
67     return data;
68 }
69
70 static void PutImageBytes(HXLBMPFILE& bmp, int width, int height,
71     const std::vector<RGBQUAD>& palette,
72     const std::vector<BYTE>& data) {
73     if (data.size() != static_cast<size_t>(width) * height) {
74         throw std::runtime_error("Decoded byte count does not match image size.");

```

```

75     }
76
77     bmp.iImagew = width;
78     bmp.iImageh = height;
79     bmp.iYRGBnum = 1;
80     if (!bmp.IsImageDataOk()) {
81         throw std::runtime_error("Failed to allocate decoded image.");
82     }
83
84     for (int i = 0; i < 256; ++i) {
85         bmp.rgbPalette[i] = palette.empty() ? RGBQUAD{static_cast<BYTE>(i), static_cast<BYTE>(i), static_cast<
86                                                     <BYTE>(i), 0}
87                                     : palette[i];
88     }
89
90     size_t pos = 0;
91     for (int y = 0; y < height; ++y) {
92         BYTE* row = bmp.pDataAt(y);
93         std::copy(data.begin() + pos, data.begin() + pos + width, row);
94         pos += width;
95     }
96
97     static double CalculateEntropy(const std::vector<BYTE>& data) {
98         uint64_t hist[256] = {0};
99         for (BYTE value : data) ++hist[value];
100
101         double entropy = 0.0;
102         const double total = static_cast<double>(data.size());
103         for (uint64_t count : hist) {
104             if (count == 0) continue;
105             double p = static_cast<double>(count) / total;
106             entropy -= p * std::log(p) / std::log(2.0);
107         }
108         return entropy;
109     }
110
111     static std::vector<uint16_t> LzwEncode(const std::vector<BYTE>& data) {
112         if (data.empty()) return {};
113
114         std::unordered_map<uint64_t, uint16_t> dict;
115         dict.reserve(70000);
116         uint32_t nextCode = 256;
117
118         uint32_t prefix = data[0];
119         std::vector<uint16_t> codes;
120         codes.reserve(data.size() / 2);
121
122         for (size_t i = 1; i < data.size(); ++i) {
123             BYTE value = data[i];
124             uint64_t key = MakeKey(prefix, value);
125             auto it = dict.find(key);
126             if (it != dict.end()) {
127                 prefix = it->second;
128             } else {
129                 codes.push_back(static_cast<uint16_t>(prefix));
130                 if (nextCode <= kMaxCode) {
131                     dict.emplace(key, static_cast<uint16_t>(nextCode++));

```



```

132     }
133     prefix = value;
134 }
135 }
136
137 codes.push_back(static_cast<uint16_t>(prefix));
138 return codes;
139 }
140
141 struct DecodeEntry {
142     int prefix;
143     BYTE value;
144 };
145
146 static std::vector<BYTE> ExpandCode(const std::vector<DecodeEntry>& dict, uint32_t code) {
147     if (code >= dict.size()) {
148         throw std::runtime_error("Invalid LZW code in compressed stream.");
149     }
150
151     std::vector<BYTE> reversed;
152     while (code != static_cast<uint32_t>(-1)) {
153         reversed.push_back(dict[code].value);
154         code = static_cast<uint32_t>(dict[code].prefix);
155     }
156     std::reverse(reversed.begin(), reversed.end());
157     return reversed;
158 }
159
160 static std::vector<BYTE> LzwDecode(const std::vector<uint16_t>& codes, size_t expectedSize) {
161     if (codes.empty()) return {};
162
163     std::vector<DecodeEntry> dict;
164     dict.reserve(70000);
165     for (int i = 0; i < 256; ++i) {
166         dict.push_back({-1, static_cast<BYTE>(i)});
167     }
168
169     std::vector<BYTE> output;
170     output.reserve(expectedSize);
171
172     uint32_t prevCode = codes[0];
173     std::vector<BYTE> prevSeq = ExpandCode(dict, prevCode);
174     output.insert(output.end(), prevSeq.begin(), prevSeq.end());
175
176     for (size_t i = 1; i < codes.size(); ++i) {
177         uint32_t code = codes[i];
178         std::vector<BYTE> seq;
179
180         if (code < dict.size()) {
181             seq = ExpandCode(dict, code);
182         } else if (code == dict.size()) {
183             seq = prevSeq;
184             seq.push_back(prevSeq.front());
185         } else {
186             throw std::runtime_error("Invalid LZW code order in compressed stream.");
187         }
188
189         output.insert(output.end(), seq.begin(), seq.end());

```

```

190
191     if (dict.size() <= kMaxCode) {
192         dict.push_back({static_cast<int>(prevCode), seq.front()});
193     }
194
195     prevCode = code;
196     prevSeq = std::move(seq);
197 }
198
199 if (output.size() != expectedSize) {
200     throw std::runtime_error("Decoded size differs from the original image size.");
201 }
202 return output;
203 }
204
205 static void SaveCompressedFile(const std::string& path, const EncodedFile& file) {
206     std::ofstream out(path, std::ios::binary);
207     if (!out) throw std::runtime_error("Cannot create compressed file: " + path);
208
209     out.write("LZW3", 4);
210     WriteU32(out, 1);
211     WriteU32(out, static_cast<uint32_t>(file.width));
212     WriteU32(out, static_cast<uint32_t>(file.height));
213     WriteU32(out, 1);
214     WriteU32(out, static_cast<uint32_t>(file.width * file.height));
215     WriteU32(out, static_cast<uint32_t>(file.codes.size()));
216
217     for (int i = 0; i < 256; ++i) {
218         RGBQUAD q = file.palette[i];
219         out.put(static_cast<char>(q.rgbBlue));
220         out.put(static_cast<char>(q.rgbGreen));
221         out.put(static_cast<char>(q.rgbRed));
222         out.put(static_cast<char>(q.rgbReserved));
223     }
224
225     for (uint16_t code : file.codes) {
226         WriteU16(out, code);
227     }
228 }
229
230 static EncodedFile LoadCompressedFile(const std::string& path) {
231     std::ifstream in(path, std::ios::binary);
232     if (!in) throw std::runtime_error("Cannot open compressed file: " + path);
233
234     char magic[4];
235     in.read(magic, 4);
236     if (std::string(magic, 4) != "LZW3") {
237         throw std::runtime_error("Compressed file magic is not LZW3.");
238     }
239
240     uint32_t version = ReadU32(in);
241     if (version != 1) throw std::runtime_error("Unsupported compressed file version.");
242
243     EncodedFile file;
244     file.width = static_cast<int>(ReadU32(in));
245     file.height = static_cast<int>(ReadU32(in));
246     uint32_t channels = ReadU32(in);
247     uint32_t pixelCount = ReadU32(in);

```

```

248     uint32_t codeCount = ReadU32(in);
249     if (channels != 1 || pixelCount != static_cast<uint32_t>(file.width * file.height)) {
250         throw std::runtime_error("Compressed file metadata is invalid.");
251     }
252
253     file.palette.resize(256);
254     for (int i = 0; i < 256; ++i) {
255         file.palette[i].rgbBlue = static_cast<BYTE>(in.get());
256         file.palette[i].rgbGreen = static_cast<BYTE>(in.get());
257         file.palette[i].rgbRed = static_cast<BYTE>(in.get());
258         file.palette[i].rgbReserved = static_cast<BYTE>(in.get());
259         if (!in) throw std::runtime_error("Unexpected end of file while reading palette.");
260     }
261
262     file.codes.reserve(codeCount);
263     for (uint32_t i = 0; i < codeCount; ++i) {
264         file.codes.push_back(ReadU16(in));
265     }
266     return file;
267 }
268
269 static bool SameImage(const HXLBMPFILE& a, const std::vector<BYTE>& aData,
270                     const HXLBMPFILE& b, const std::vector<BYTE>& bData) {
271     return a.iImagew == b.iImagew && a.iImageh == b.iImageh &&
272            a.iYRGBnum == b.iYRGBnum && aData == bData;
273 }
274
275 int main(int argc, char* argv[]) {
276     const std::string inputBmp = argc > 1 ? argv[1] : "b8gray.bmp";
277     const std::string compressedPath = argc > 2 ? argv[2] : "C.dat";
278     const std::string decodedBmp = argc > 3 ? argv[3] : "Id.bmp";
279
280     try {
281         HXLBMPFILE src;
282         if (!src.LoadBMPFile(const_cast<char*>(inputBmp.c_str()))) {
283             throw std::runtime_error("Failed to load input BMP: " + inputBmp);
284         }
285
286         std::vector<BYTE> srcData = GetImageBytes(src);
287         double entropy = CalculateEntropy(srcData);
288
289         EncodedFile encoded;
290         encoded.width = src.iImagew;
291         encoded.height = src.iImageh;
292         encoded.palette.assign(src.rgbPalette, src.rgbPalette + 256);
293         encoded.codes = LzwEncode(srcData);
294         SaveCompressedFile(compressedPath, encoded);
295
296         EncodedFile loaded = LoadCompressedFile(compressedPath);
297         std::vector<BYTE> decodedData = LzwDecode(
298             loaded.codes,
299             static_cast<size_t>(loaded.width) * loaded.height);
300
301         HXLBMPFILE dst;
302         PutImageBytes(dst, loaded.width, loaded.height, loaded.palette, decodedData);
303         if (!dst.SaveBMPFile(const_cast<char*>(decodedBmp.c_str()))) {
304             throw std::runtime_error("Failed to save decoded BMP: " + decodedBmp);
305         }

```

```

306
307     std::ifstream compressed(compressedPath, std::ios::binary | std::ios::ate);
308     double fileBpp = static_cast<double>(compressed.tellg()) * 8.0 / srcData.size();
309     double codeBpp = static_cast<double>(encoded.codes.size()) * 16.0 / srcData.size();
310     bool identical = SameImage(src, srcData, dst, decodedData);
311
312     std::ofstream result("result.txt");
313     std::ostream* outs[] = {&std::cout, &result};
314     for (std::ostream* os : outs) {
315         *os << std::fixed << std::setprecision(6);
316         *os << "LZW image coding experiment\n";
317         *os << "Input image: " << inputBmp << "\n";
318         *os << "Compressed file: " << compressedPath << "\n";
319         *os << "Decoded image: " << decodedBmp << "\n";
320         *os << "Image size: " << src.iImagew << " x " << src.iImageh << "\n";
321         *os << "Original pixels: " << srcData.size() << "\n";
322         *os << "LZW code count: " << encoded.codes.size() << "\n";
323         *os << "Encoded bpp (codes only): " << codeBpp << "\n";
324         *os << "Encoded bpp (C.dat file): " << fileBpp << "\n";
325         *os << "Original entropy: " << entropy << " bit/pixel\n";
326         *os << "Is == Id: " << (identical ? "YES" : "NO") << "\n";
327     }
328
329     return identical ? 0 : 2;
330 } catch (const std::exception& ex) {
331     std::cerr << "Error: " << ex.what() << "\n";
332     return 1;
333 }
334 }
```